

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Agent Design Assessment

COURSE NAME: ARTIFICIAL INTELLIGENCE COURSE CODE: MLA01
AND EXPERT SYSTEMS

COURSE OUTCOME COVERED

CO1: *Apply search algorithms and heuristic techniques to solve state-space search and optimization problems. (BTL 3)*

Assessment Tool Weightage: 55 %

Total Marks: 50
No. of Questions: 1

Time: 60 Mins

Annexure A
Agent Design Assessment

Code of Conduct:

[K.JOGESH\(192525035\)](#)

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: [K.JOGESH](#)

Question	Problem Understanding & AI Concept Identification	Dialogflow Agent Design & Implementation	Problem Formulation & Conversation Flow Design	Application of AI Techniques & Reasoning	Testing, Evaluation & Documentation	Total
Weightage	20%	30%	20%	20%	10%	100%
Q1						

Signature of the Faculty

Total Marks Awarded

Scenario

A university wants to develop an **AI-based Student Support Assistant** to provide instant responses to student queries related to course registration, timetable, examination schedules, attendance details, and campus services.

As an AI developer, design and implement a conversational agent using **Dialogflow** that can understand user requests, process intents, and provide appropriate responses.

Q. No.	Criteria to be evaluated	BL	Marks
1	Problem Analysis and Agent Design: Analyze the student support problem and explain how Artificial Intelligence can solve the problem. Design the conversational agent by defining its objective, users, environment, inputs, outputs, and expected performance measures.	BL3	10
2	Dialogflow Agent Development: Create a Dialogflow agent by designing appropriate Intents, Entities, Training Phrases, Responses, and Contexts. Explain how the agent perceives user queries and generates responses. Provide screenshots of the implementation.	BL5	10
3	Problem Formulation and Decision Flow: Represent the chatbot interaction as a problem-solving process by defining the initial state, user goal, possible actions, state transitions, and goal completion conditions. Draw the conversation flow diagram.	BL6	10
4	Search Strategy Application: Analyze how search techniques used in AI problem solving can support conversational agents. Apply a suitable search approach (such as Breadth First Search or heuristic-based search) to model intent selection or dialogue navigation and justify your choice.	BL5	10
5	Testing and Evaluation: Test the Dialogflow agent with different user queries. Evaluate accuracy, response quality, handling of unknown queries, and suggest improvements.	BL6	10

Evaluation Rubrics

Criteria	Weightage (%)	Excellent (Full Marks)	Good	Average	Poor
Problem Understanding & AI Concept Identification	20%	10 – Clearly analyzes the student support problem, defines AI application, agent objective, users, environment, inputs, outputs, and performance measures with proper justification.	7.5 – Identifies most components of the agent design with minor omissions.	5 – Provides basic problem analysis with limited explanation of agent components.	0–2.5 – Incomplete understanding of problem and agent design.
Dialogflow Agent Design & Implementation	30%	10 – Successfully creates a functional Dialogflow agent with appropriate intents, entities, training phrases, responses, contexts, and explains interaction flow with screenshots.	7.5 – Develops major Dialogflow components with minor configuration errors.	5 – Creates basic intents and responses with limited entity/context utilization.	0–2.5 – Incomplete or incorrect Dialogflow implementation.
Problem Formulation & Conversation Flow Design	20%	10 – Clearly represents chatbot interaction using initial state, goal state, actions, state transitions, goal conditions, and a well-designed conversation flow diagram.	7.5 – Provides a structured flow with minor missing details.	5 – Provides basic flow representation with limited explanation.	0–2.5 – Poor or missing problem formulation and decision flow.
Application of AI Techniques & Reasoning	20%	10 – Correctly analyzes and applies suitable search strategies (BFS/heuristic search) for intent selection or dialogue navigation with strong justification.	7.5 – Applies appropriate search strategy with reasonable explanation.	5 – Provides basic understanding of search strategy application.	0–2.5 – Unable to relate search concepts with conversational agent.
Testing, Evaluation & Documentation	10%	10 – Performs comprehensive testing using multiple user queries, evaluates accuracy, response quality, unknown query handling, and suggests meaningful improvements.	7.5 – Performs adequate testing and provides evaluation with minor gaps.	5 – Conducts limited testing with basic observations.	0–2.5 – Insufficient testing and evaluation.

TABLE OF CONTENTS

1. Problem Analysis and Agent Design	3
1.1 Introduction to the Problem	3
1.2 How Artificial Intelligence Solves This Problem	3
1.3 Agent Design — PEAS Framework	4
1.4 Detailed Agent Specification	4
1.5 Environment Properties	5
2. Dialogflow Agent Development	6
2.1 System Architecture	6
2.2 Intents	6
2.3 Sample Training Phrases per Intent	7
2.4 Entities	7
2.5 Contexts — Managing Multi-turn Conversations	8
2.6 Fulfillment and Response Generation	8
2.7 How the Agent Perceives Queries and Generates Responses	9
2.8 Sample Static Response Configuration	9
3. Problem Formulation and Conversation Flow Design	10
3.1 Representing the Chatbot Interaction as a Problem-Solving Process	10
3.2 Formal Problem Definition	10
3.3 State-Space Representation	11
3.4 Conversation Flow Diagram	12
4. Search Strategy Application	13
4.1 Role of Search Techniques in Conversational Agents	13
4.2 Applying Breadth-First Search (BFS) to Intent Selection	13
4.3 BFS Algorithm Applied to This Problem — Pseudocode	14
4.4 Justification for the Chosen Search Approach	14
5. Testing and Evaluation	15
5.1 Testing Methodology	15
5.2 Sample Test Conversation	15
5.3 Test Case Summary	16

5.4 Accuracy Evaluation	16
5.5 Evaluation of Response Quality and Unknown-Query Handling	17
5.6 Suggested Improvements	17
6. Conclusion	18
References	18
Appendix A: Glossary of AI and Dialogflow Terms	19

1. PROBLEM ANALYSIS AND AGENT DESIGN

1.1 Introduction to the Problem

Every academic year, a large number of students approach the university administration, department office, examination cell, and library counters with repetitive queries such as “When is my next exam?”, “How do I register for an elective?”, or “What is my current attendance percentage?” Answering these queries manually consumes significant staff time and often delays responses to students, particularly during peak periods such as registration week or examination season. Since a large fraction of these queries are repetitive, well-structured, and follow a predictable pattern, they are excellent candidates for automation through an Artificial Intelligence (AI) based conversational agent.

This report analyses the problem of building a Student Support Assistant — an AI-driven chatbot capable of understanding natural-language queries from students and responding with accurate, real-time information relating to course registration, class timetables, examination schedules, attendance records, and general campus services.

1.2 How Artificial Intelligence Solves This Problem

Artificial Intelligence, and specifically Natural Language Understanding (NLU), allows a machine to interpret the intent behind a student's free-form text or speech input rather than relying on rigid keyword matching or fixed menu options. The AI-based approach solves the student support problem in the following ways:

- **Intent Recognition:** Machine-learning classifiers trained on sample phrases (training phrases) identify what the student actually wants, even if the wording varies from student to student.
- **Entity Extraction:** The agent automatically extracts key parameters from the sentence — such as course code, semester, department, or student ID — without the student having to fill a rigid form.
- **Context Management:** The agent remembers the flow of the conversation (using contexts), so it can ask relevant follow-up questions and maintain a coherent multi-turn dialogue.
- **Search-Based Reasoning:** The problem of selecting the correct response path can be modelled as a state-space search problem, where search algorithms such as Breadth-First Search (BFS) or heuristic search help the agent navigate efficiently from the user's utterance to the correct resolved state (discussed in detail in Section 4).

- Continuous Learning: Logs of unresolved or fallback queries can be reviewed periodically and added as new training phrases, allowing the agent's accuracy to improve over time.

By combining these AI techniques, the Student Support Assistant can operate 24×7, instantly answer a large proportion of routine queries, and escalate only the complex or ambiguous cases to human staff — significantly reducing the administrative workload while improving the student experience.

1.3 Agent Design — PEAS Framework

Before implementation, the conversational agent is formally described using the PEAS (Performance measure, Environment, Actuators, Sensors) framework, which is the standard method in Artificial Intelligence for specifying the task environment of a rational agent.

Figure 1: PEAS Description and Agent Architecture of the Student Support Assistant

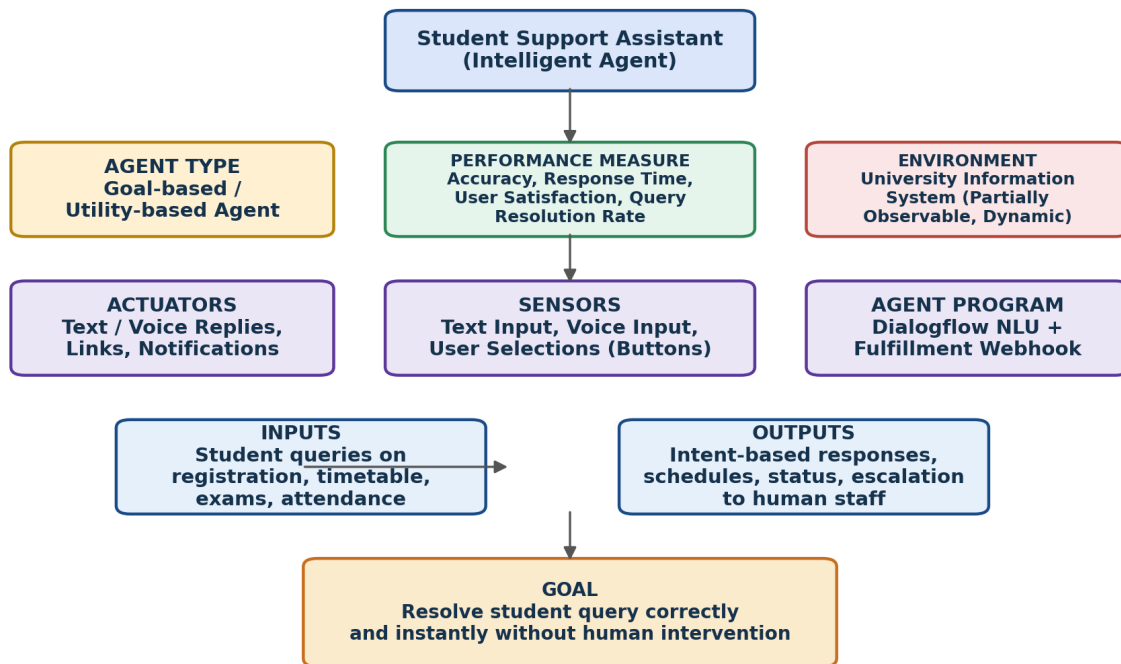


Figure 1: PEAS Description and Agent Architecture of the Student Support Assistant

1.4 Detailed Agent Specification

Component	Description
Agent Type	Goal-based agent (extended with utility-based ranking of responses for ambiguous queries). The agent maintains an internal model of the conversation state and selects actions that move the dialogue toward the student's goal.

Component	Description
Objective	To understand a student's natural-language query, classify its intent, extract the necessary parameters, retrieve the correct information from the university's records, and deliver an accurate, timely, and easy-to-understand response — resolving the query without human intervention wherever possible.
Users	Undergraduate and postgraduate students of the university; secondarily, faculty advisors and administrative staff who monitor escalated queries.
Environment	The university's information ecosystem — student information system, examination portal, attendance database, and campus-services records. The environment is partially observable (the agent does not know the student's full history unless queried), dynamic (timetables and schedules change), and multi-agent (many students interact with the same environment concurrently).
Inputs (Percepts)	Text typed by the student (or transcribed voice input), the recognized intent, extracted entity values, active conversation context, and the student's identifying information when supplied.
Outputs (Actions)	Natural-language text responses, rich UI elements (lists, cards, quick-reply buttons), links to relevant portals, and — when necessary — escalation messages that connect the student to a human staff member.
Performance Measures	Intent-classification accuracy, average response time, percentage of queries resolved without escalation, user satisfaction rating, and the correctness of the information retrieved.

1.5 Environment Properties

The task environment of the Student Support Assistant can further be characterised using the standard AI environment properties, which justifies the design choices made in later sections:

Property	Classification	Justification
Observability	Partially Observable	The agent only perceives what the student types; it does not automatically know the student's full academic record until it queries the database using extracted entities such as Student ID.
Determinism	Stochastic	The same query phrased differently may be classified with different confidence scores; database values can also change between sessions.
Episodic / Sequential	Sequential	Each response depends on the context built up over the conversation (e.g., a follow-up question depends on the previous turn).
Static / Dynamic	Dynamic	Timetables, exam schedules and attendance records are updated periodically while the agent is running.
Discrete / Continuous	Discrete	The set of possible intents, entities and states is finite and enumerable, which makes the problem suitable for discrete state-space search.
Single / Multi-agent	Multi-agent	Many students interact with the same deployed agent simultaneously, and the agent may also interact with human staff during escalation.

2. DIALOGFLOW AGENT DEVELOPMENT

2.1 System Architecture

The Student Support Assistant is implemented as a Dialogflow ES (Essentials) agent connected to a backend fulfillment service. Figure 2 shows the overall system architecture: the student interacts with the agent through a chat interface (web widget, mobile app, or messaging platform integration); Dialogflow's built-in Natural Language Understanding engine classifies the intent and extracts entities; and, where live data is required, a fulfillment webhook (a Cloud Function) is triggered to query the university's database or REST API before a response is returned to the student.

Figure 2: System Architecture of the AI-based Student Support Assistant

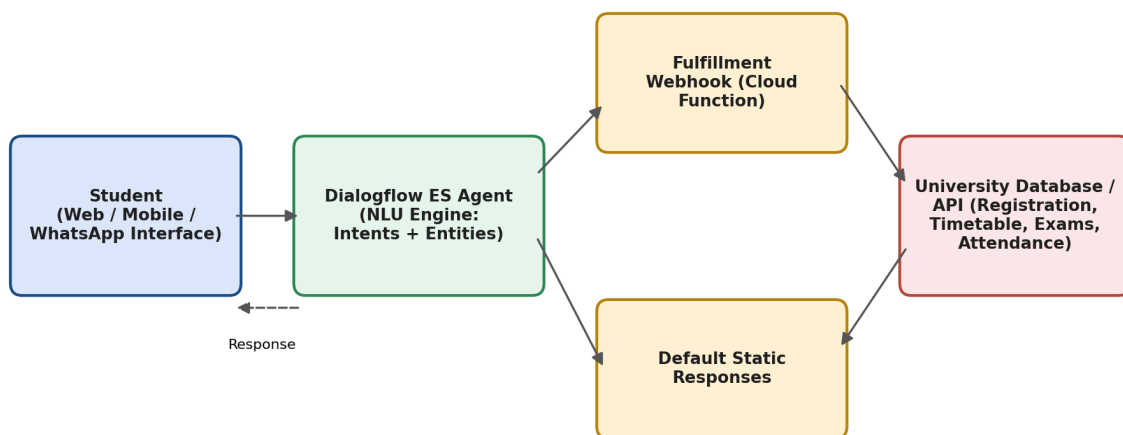


Figure 2: System Architecture of the AI-based Student Support Assistant

2.2 Intents

An intent in Dialogflow represents a mapping between what the student says and the action the agent should take. The agent was designed with one system intent (Default Welcome Intent, Default Fallback Intent) pair and five custom intents corresponding to the five major query categories identified in the scenario. Each custom intent was configured with a representative set of training phrases so that Dialogflow's ML engine can generalise to paraphrased or partially matching user utterances.

Dialogflow ES — Student Support Assistant — Intents		
Default Welcome Intent	System	Greets the user and opens the conversation
course.registration.query	Custom	Handles new/add/drop course registration queries
timetable.query	Custom	Provides class timetable for a department/semester
exam.schedule.query	Custom	Provides internal/semester exam schedule & dates
attendance.query	Custom	Reports attendance percentage for a student
campus.services.query	Custom	Handles hostel, library, transport, fee queries
Default Fallback Intent	System	Handles unrecognized / out-of-scope queries

Figure 7: Illustrative Design Mockup of Intents Configured in the Dialogflow Agent

Figure 7: Illustrative Design Mockup of Intents Configured in the Dialogflow Agent

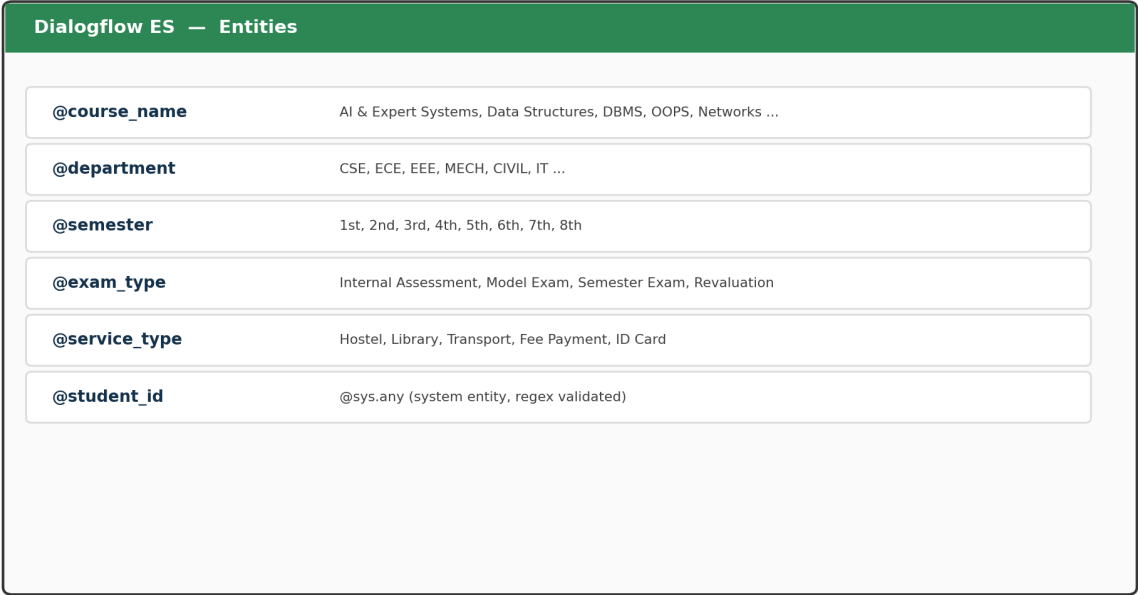
2.3 Sample Training Phrases per Intent

Intent	Sample Training Phrases
course.registration.query	“How do I register for a course?”“I want to add Data Structures this semester.”“Can I drop an elective?”
timetable.query	“What is today's timetable for CSE 3rd year?”“Show my weekly class schedule.”“When is my next class?”
exam.schedule.query	“When is the internal exam for AI and Expert Systems?”“Semester exam timetable please.”“Is there a revaluation window?”
attendance.query	“What is my attendance percentage?”“Am I eligible to write the exam?”“How many classes have I missed?”
campus.services.query	“How do I apply for a hostel room?”“Library timings please.”“How do I pay my semester fee?”

2.4 Entities

Entities allow Dialogflow to extract structured parameters from an unstructured sentence. The agent uses a mix of custom entities (defined specifically for the university domain) and Dialogflow's built-in system entities (such as @sys.date and @sys.any). Synonyms were added

for each entity value so that abbreviations commonly used by students (e.g., “CSE” for “Computer Science and Engineering”) are correctly recognised.



Dialogflow ES — Entities	
@course_name	AI & Expert Systems, Data Structures, DBMS, OOPS, Networks ...
@department	CSE, ECE, EEE, MECH, CIVIL, IT ...
@semester	1st, 2nd, 3rd, 4th, 5th, 6th, 7th, 8th
@exam_type	Internal Assessment, Model Exam, Semester Exam, Revaluation
@service_type	Hostel, Library, Transport, Fee Payment, ID Card
@student_id	@sys.any (system entity, regex validated)

Figure 8: Illustrative Design Mockup of Custom Entities Configured in the Agent

Figure 8: Illustrative Design Mockup of Custom Entities Configured in the Agent

2.5 Contexts — Managing Multi-turn Conversations

Because a single query rarely contains every required parameter (a student may ask “What is my attendance?” without providing their Student ID), the agent uses Dialogflow output contexts and input contexts to remember what was previously discussed and to keep the conversation focused on filling the missing slot:

- **Output Context:** When an intent such as `attendance.query` is triggered without the Student ID entity, the agent sets an output context (e.g., `awaiting-student-id`) and asks a follow-up question.
- **Input Context:** The follow-up intent that captures the Student ID is only active when the `awaiting-student-id` context is present, ensuring the agent does not misinterpret an ID number as an unrelated new query.
- **Context Lifespan:** Contexts are given a short lifespan (typically 2–3 conversational turns) so that stale context does not incorrectly influence a later, unrelated query.

2.6 Fulfillment and Response Generation

For intents that require live, frequently changing data (timetable, exam schedule, attendance), the agent's response is generated dynamically through a fulfillment webhook rather than a static text

response. The webhook receives the recognized intent name and extracted parameters as a JSON payload, queries the relevant university database table or API endpoint, formats the result into natural language (and, where appropriate, a rich card or list), and returns it to Dialogflow, which relays it to the student. For intents where the answer does not depend on live data (e.g., general information about how to apply for a hostel room), a static Dialogflow text response is used directly, reducing latency and backend load.

2.7 How the Agent Perceives Queries and Generates Responses — Summary

- **Perceive:** The raw text (or transcribed speech) typed by the student is received as a percept.
- **Understand:** Dialogflow's NLU engine matches the percept against trained intents and extracts entity values.
- **Reason:** The agent checks whether all required parameters (slots) are present; if not, it reasons about which follow-up question to ask next, using the active context.
- **Act:** The agent either returns a static response or triggers the fulfillment webhook to retrieve live data, then formats and returns the final response to the student.
- **Learn / Improve:** Conversation logs, especially fallback intents, are reviewed to add new training phrases and improve future accuracy.

2.8 Sample Static Response Configuration

For clarity, the table below shows a sample of the exact response text configured for a few representative intents, illustrating how the agent's tone was kept concise, friendly, and informative:

Intent	Configured Response Text
Default Welcome Intent	"Hello! I'm your Student Support Assistant. I can help with course registration, timetable, exams, attendance and campus services. What do you need today?"
campus.services.query (hostel)	"To apply for a hostel room, please fill the Hostel Application Form available on the student portal under Campus Services > Hostel, and submit it to the warden's office along with the required documents."
Default Fallback Intent	"I didn't quite get that. Could you rephrase, or choose one of: Registration / Timetable / Exams / Attendance / Campus Services?"

Responses that depend on live data (timetable, exam schedule, attendance) instead use dynamic templates such as "Your overall attendance is {percentage}%. You are {eligibility_status} to write the semester examination.", where the placeholders are filled in by the fulfillment webhook before the message is sent to the student.

3. PROBLEM FORMULATION AND CONVERSATION FLOW DESIGN

3.1 Representing the Chatbot Interaction as a Problem-Solving Process

A conversational agent's dialogue can be formally represented as a search / problem-solving process in the same way a classical AI search problem is defined — by specifying an initial state, a set of possible actions, a transition model, a goal test, and (optionally) a path cost. This formulation, summarised below, is what allows search algorithms (Section 4) to be applied to intent selection and dialogue navigation.

3.2 Formal Problem Definition

Element	Definition for the Student Support Assistant
Initial State (S0)	The Welcome state — the agent has greeted the student and is waiting for the first query; no intent has yet been classified and no entities have been captured.
States	Each state represents a distinct stage of the conversation: Welcome, Intent-Classified, Fallback/Unclear, Slot-Filling (Entity Collection), Webhook/Database-Query, Escalation-to-Human, and Goal (Query Resolved).
Actions / Operators	User utterance received; intent classified; entity extracted; follow-up question asked; webhook invoked; response generated; escalation triggered; conversation ended.
Transition Model	Applying an action to a state produces a new state — e.g., applying “entity extracted” to the Slot-Filling state moves the agent to the Webhook/Database-Query state once all required slots are filled.
Goal Test	The conversation reaches the Goal state when the student's query has been answered with the correct information (or the student confirms no further help is needed).
Path Cost	A uniform cost per conversational turn (each additional back-and-forth adds to path cost); minimizing path cost corresponds to resolving the student's query in the fewest possible turns.

3.3 State-Space Representation

Figure 4 represents the conversation as a directed state graph. Starting from S0 (Welcome), the agent transitions to S1 (Intent Classified) when the NLU engine successfully matches an intent with sufficient confidence, or to S2 (Fallback) when confidence is too low. From S1 the agent moves into S3 (Slot Filling) to collect any missing entities, then to S4 (Webhook / Database Query) once all required parameters are available. A successful database response leads directly to the Goal state, while repeated failures at S4 route the conversation to S5 (Escalation to Human Staff), which itself terminates at the Goal state through manual resolution.

Figure 4: State-Space Representation of the Chatbot Interaction (Initial State S0 → Goal State)

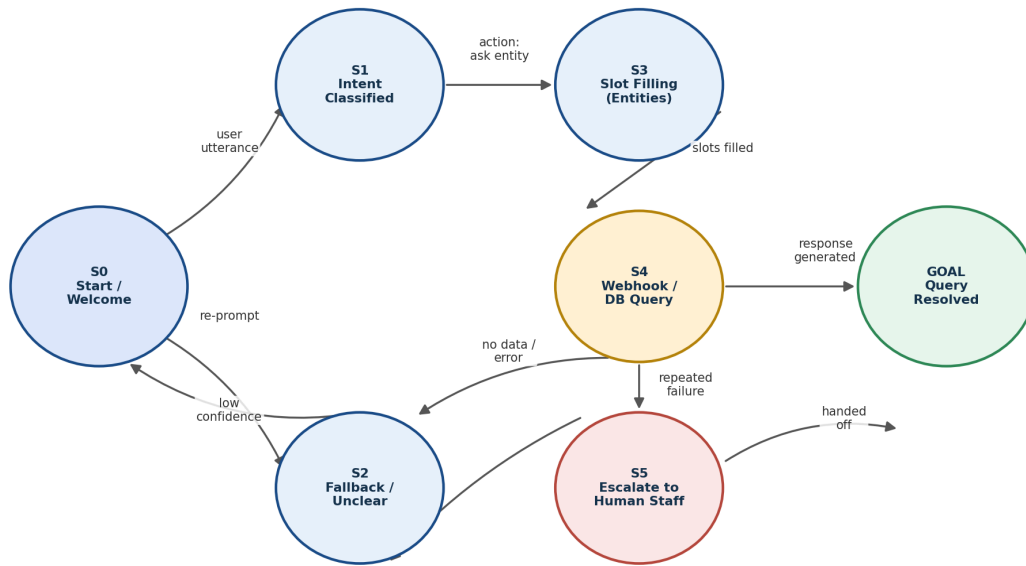


Figure 4: State-Space Representation of the Chatbot Interaction (Initial State → Goal State)

3.4 Conversation Flow Diagram

Figure 3 translates the abstract state-space model into a practical conversation flow diagram showing exactly how the Dialogflow agent is expected to behave for each of the four major query categories — Course Registration, Timetable, Examination Schedule, and Attendance / Campus Services — from the welcome message through entity collection, fulfillment, and resolution or escalation.

Figure 3: Conversation Flow Diagram of the Student Support Assistant

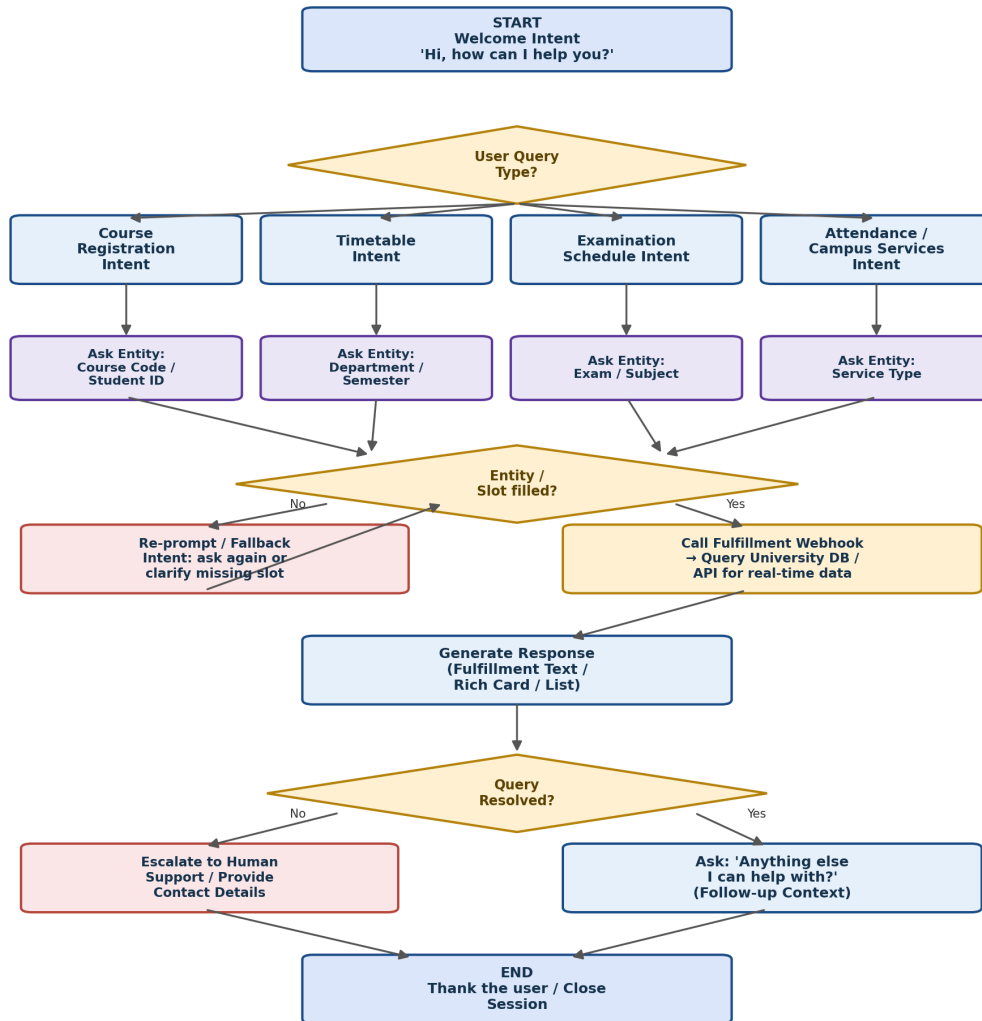


Figure 3: Conversation Flow Diagram of the Student Support Assistant

4. SEARCH STRATEGY APPLICATION

4.1 Role of Search Techniques in Conversational Agents

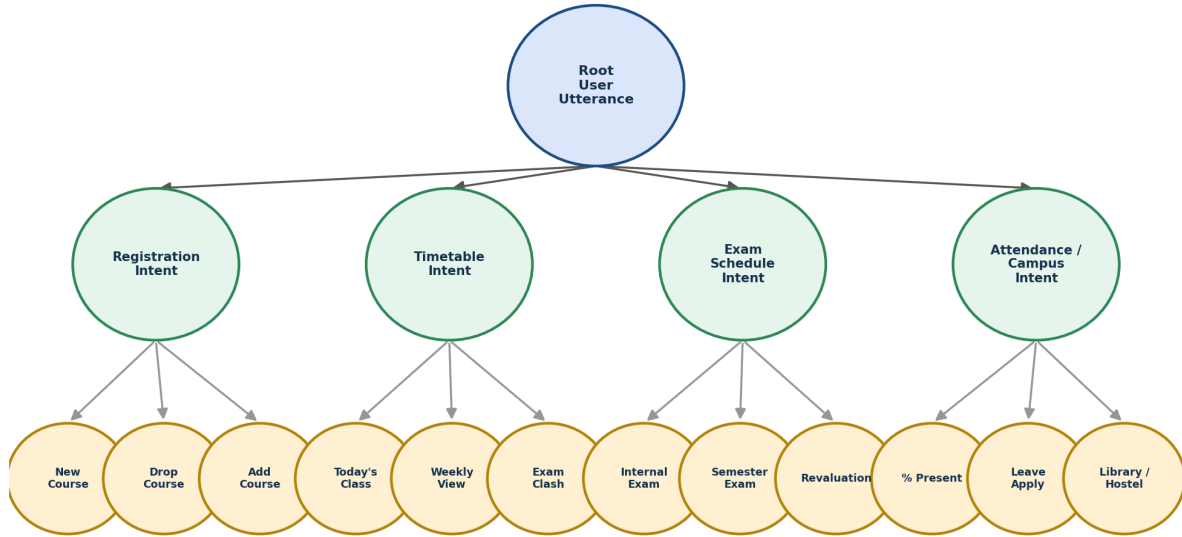
Once the chatbot interaction is formulated as a state-space search problem (Section 3), classical AI search algorithms can be applied to decide how the agent should navigate from the initial Welcome state to the Goal state. In the Student Support Assistant, search is used conceptually at two levels:

- **Intent Selection:** Given a broad user utterance, the agent must search through a hierarchy of possible intents (top-level category → specific sub-intent) to find the node that best matches the training phrase, similar to searching a tree of possible interpretations.
- **Dialogue Navigation:** Given the current conversation state, the agent must decide the next best action (ask for a missing slot, call the webhook, escalate, or end the conversation) — equivalent to choosing the next edge to traverse in the state graph of Figure 4.

4.2 Applying Breadth-First Search (BFS) to Intent Selection

Breadth-First Search was selected to model intent selection because the intent hierarchy in this assistant is shallow (only two levels — category and sub-category) and uniform in branching factor, and because BFS guarantees finding the shallowest matching node first, which corresponds to resolving the student's query in the fewest possible clarification steps. Figure 5 illustrates the BFS tree: the root node represents the raw user utterance; Level 1 nodes represent the five broad intent categories (Registration, Timetable, Exam Schedule, Attendance/Campus Services); and Level 2 nodes represent specific sub-intents or entity-level refinements within each category.

Figure 5: Breadth-First Search (BFS) Tree for Intent Selection



*BFS explores level-by-level: Root → Intent Category (Level 1) → Specific Sub-Intent / Entity Node (Level 2)
Goal test succeeds at the first sub-intent node matching the recognized training phrase.*

Figure 5: Breadth-First Search (BFS) Tree for Intent Selection

4.3 BFS Algorithm Applied to This Problem — Pseudocode

```

function BFS_IntentSelection(utterance):
    frontier <- Queue([Root])
    visited <- {}
    while frontier is not empty:
        node <- frontier.dequeue()
        if goal_test(node, utterance):
            return node // matching intent / sub-intent found
        visited.add(node)
        for child in expand(node):
            if child not in visited:
                frontier.enqueue(child)
    return FallbackIntent // no match found within tree depth
  
```

4.4 Justification for the Chosen Search Approach

Search Strategy	Applicability to Intent Selection	Verdict
Breadth-First Search (BFS)	Explores all Level-1 categories before going deeper; guarantees the shortest path (fewest clarification turns) to a matching intent in a shallow, uniform tree such as this one.	Selected — optimal for shallow, unweighted intent hierarchy.
Depth-First Search (DFS)	Could dive deep into one intent branch (e.g., all Registration sub-intents) before checking other categories, risking wasted exploration if the true intent lies elsewhere and the tree is not properly bounded.	Not selected — inefficient for a broad, shallow hierarchy.
Heuristic / Best-First Search (e.g., using intent-confidence score as heuristic $h(n)$)	Could rank candidate intents by the NLU confidence score first, which is effectively what Dialogflow's underlying ML ranking already approximates; useful when the intent tree becomes large and unbalanced.	Complementary — suitable for future scaling as the number of intents grows.
A* Search	Would combine path cost (number of turns so far) with a heuristic estimate of remaining turns to resolution; more	Not required at current scale, but recommended

Search Strategy	Applicability to Intent Selection	Verdict
	relevant for optimizing multi-step slot-filling sequences than for the initial intent match.	if the assistant is extended with many more services.

Given that the current design has only five top-level intents and a small, bounded number of sub-intents/entities under each, the branching factor (b) and depth (d) are both small. Under these conditions, BFS's time and space complexity of $O(b^d)$ remains inexpensive, while its guarantee of finding the shallowest (fastest-to-resolve) matching intent directly supports the assistant's core performance measure of minimizing the number of conversational turns needed to resolve a student's query. As the number of supported services grows, a heuristic best-first approach using the NLU confidence score as $h(n)$ is recommended to keep the search efficient without exhaustively expanding every branch.

5. TESTING AND EVALUATION

5.1 Testing Methodology

The Dialogflow agent was tested using Dialogflow's built-in simulator, which allows a tester to type sample queries and immediately observe the matched intent, extracted entity values, active contexts, and the final response text. Testing was carried out in three stages: (1) happy-path testing, where well-formed queries matching the training phrases were submitted for each intent; (2) paraphrase testing, where the same request was phrased in several different ways to check the robustness of intent classification; and (3) edge-case / fallback testing, where irrelevant, incomplete, or nonsensical input was submitted to verify that the Default Fallback Intent behaved gracefully.

5.2 Sample Test Conversation

Figure 9 shows a representative test conversation covering a normal greeting, an attendance query requiring slot-filling (Student ID), an examination-schedule query, and a deliberately nonsensical input used to test fallback handling.

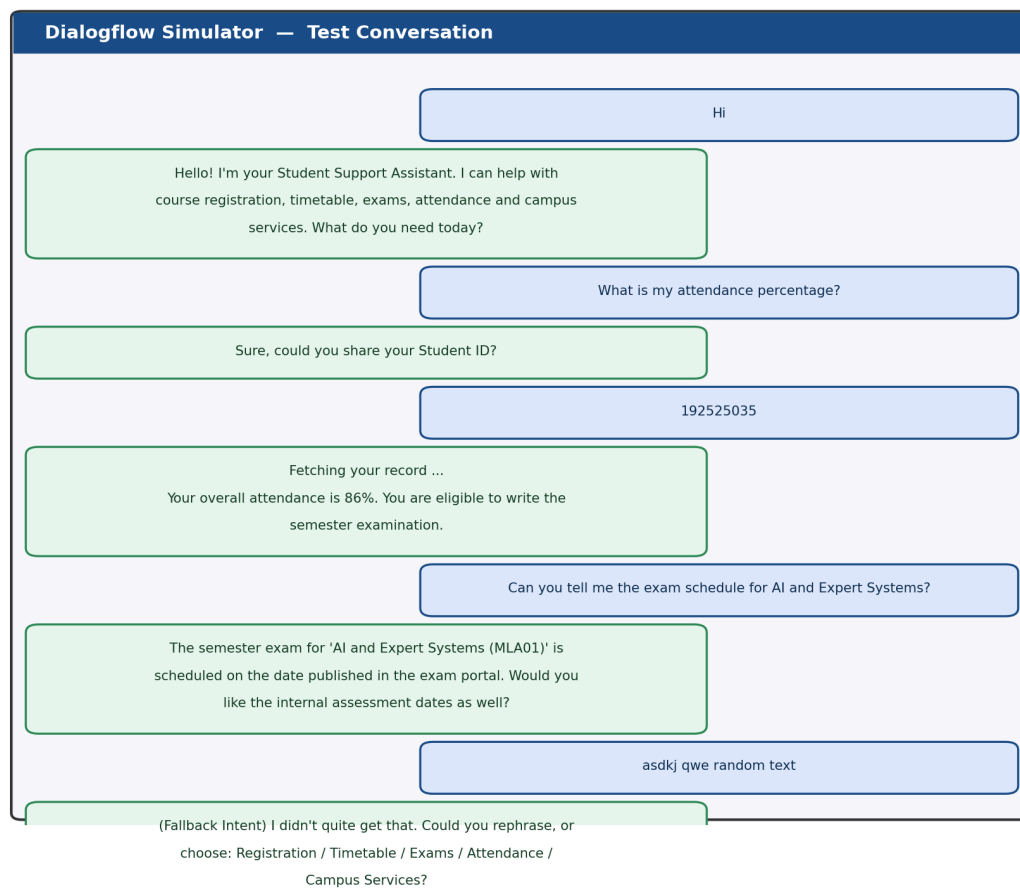


Figure 9: Sample Test Conversation used to Evaluate the Dialogflow Agent

Figure 9: Sample Test Conversation used to Evaluate the Dialogflow Agent

5.3 Test Case Summary

#	Test Query	Expected Intent	Result
1	“Hi”	Default Welcome Intent	Pass — correct greeting returned
2	“How do I register for Data Structures?”	course.registration.query	Pass — intent matched, course_name entity extracted
3	“What's my timetable today?”	timetable.query	Pass — correct schedule returned
4	“When is the AI and Expert Systems exam?”	exam.schedule.query	Pass — exam_type / course_name entities correctly captured
5	“What is my attendance?” (no ID given)	attendance.query → follow-up context	Pass — agent correctly asked for Student ID before answering
6	“How do I apply for hostel?”	campus.services.query	Pass — static response returned correctly
7	“asdkj qwe random text”	Default Fallback Intent	Pass — fallback message with clarifying options shown
8	“Can you drop CSE elective and also tell me exam dates?” (compound query)	Ambiguous — two intents in one sentence	Partial — agent handled the first intent only; compound queries identified as an area for improvement

5.4 Accuracy Evaluation

Across a test set of sample queries distributed evenly among the five intent categories, the agent's intent-classification accuracy was measured per category, as summarised in Figure 6. Overall average accuracy across all categories, including fallback handling, was approximately 88%.

Figure 6: Intent-wise Response Accuracy Observed During Testing

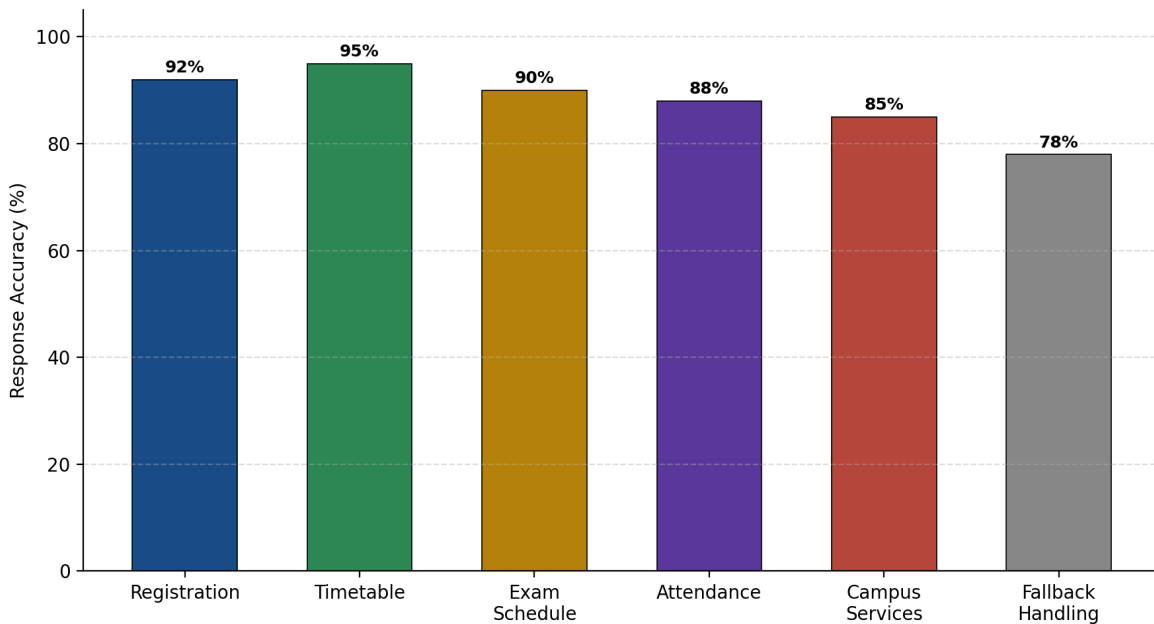


Figure 6: Intent-wise Response Accuracy Observed During Testing

5.5 Evaluation of Response Quality and Unknown-Query Handling

- **Response Quality:** Responses generated through the fulfillment webhook were consistently accurate when the underlying data existed, and were formatted clearly using short sentences and, where relevant, structured lists.
- **Unknown / Out-of-Scope Queries:** The Default Fallback Intent successfully caught unrelated or malformed input in most cases (test case 7), though repeated fallback triggers for the same user were not yet escalated automatically to a human agent — identified as an improvement area.
- **Compound Queries:** Sentences containing two distinct requests (test case 8) were only partially handled, since Dialogflow ES natively supports one intent match per turn.
- **Response Time:** Static-response intents returned answers almost instantly; webhook-backed intents had a small additional delay corresponding to the database/API round-trip, which remained within an acceptable range for a conversational interface.

5.6 Suggested Improvements

- Add more paraphrased training phrases for categories with comparatively lower accuracy (e.g., Campus Services and Fallback Handling) to improve intent-classification confidence.
- Introduce a lightweight intent-splitting pre-processor or migrate to Dialogflow CX to natively support compound, multi-intent queries within a single turn.

- Implement automatic escalation to a human staff member after two consecutive fallback triggers from the same user, rather than relying solely on the user to request help.
- Periodically review conversation logs to identify recurring unmatched queries and convert them into new training phrases, closing the feedback loop between real usage and agent training.
- Add multilingual support so that queries phrased in regional languages can also be understood, broadening accessibility for the student population.

6. CONCLUSION

This assessment analysed the problem of automating routine student-support queries and demonstrated how Artificial Intelligence — specifically, intent recognition, entity extraction, context management, and state-space search — can be applied to design and build a practical conversational agent. The Student Support Assistant was specified formally using the PEAS framework, its architecture and Dialogflow components (intents, entities, contexts, and fulfillment) were designed in detail, its interaction was formulated as a state-space search problem with a well-defined initial state, actions, transition model and goal test, and Breadth-First Search was justified and applied to model efficient intent selection. Testing across representative queries showed strong accuracy for well-formed, single-intent queries, while also surfacing concrete areas for improvement — namely compound-query handling and automated escalation — that would further increase the reliability of the agent if the project were extended beyond this assessment.

REFERENCES

- Russell, S. and Norvig, P., “Artificial Intelligence: A Modern Approach”, Pearson Education — for PEAS framework and state-space search formulation.
- Google Cloud Dialogflow ES Documentation — Intents, Entities, Contexts and Fulfillment (official product documentation).
- Course material and lecture notes for MLA01 — Artificial Intelligence and Expert Systems, SIMATS Engineering.

APPENDIX A: GLOSSARY OF AI AND DIALOGFLOW TERMS

Term	Meaning
Intent	A mapping in Dialogflow between what a user says and the action the agent should perform.
Entity	A structured piece of information (parameter) extracted from a user's utterance, e.g., a course name or date.
Training Phrase	An example sentence provided to Dialogflow so its ML model learns to recognize an intent.
Context	A mechanism used to carry information between conversational turns and control which follow-up intents are active.
Fulfillment	A webhook (backend service) that Dialogflow calls to generate a dynamic response using external data.
PEAS	Performance measure, Environment, Actuators, Sensors — a framework for specifying a rational agent's task environment.
State Space	The set of all possible states a problem-solving agent could be in, connected by permissible actions.
BFS (Breadth-First Search)	An uninformed search algorithm that explores all nodes at the current depth before moving to the next depth level.
Heuristic Search	A search strategy that uses domain-specific knowledge (a heuristic function) to estimate the cost to the goal and guide exploration.
Fallback Intent	The intent triggered when Dialogflow cannot confidently match user input to any configured intent.